Welche Datenstruktur wann?

Nachschlagetabelle für die Klausur

1 Der Entscheidungsweg

Zwei Fragen, dann steht die Antwort

```
FRAGE 1: Welche Familie?

   Lege ich Objekte unter einem Schluessel ab
   und suche spaeter darueber?          ->  MAP

   Sonst: Duerfen Duplikate vorkommen?
                        ja  ->  LIST
                        nein ->  SET

   Sonderfall: nacheinander abarbeiten  ->  QUEUE


FRAGE 2: Welche Reihenfolge?   (nur bei Map und Set)

   egal            ->  Hash...
   wie eingefuegt  ->  LinkedHash...
   sortiert        ->  Tree...
```

Bei List entfällt Frage 2 – ArrayList behält immer die Einfügereihenfolge.

Der wichtigste Satz

Du entscheidest das nicht aus dem Bauch, sondern der Aufgabentext sagt es dir. Steht dort keine Anforderung, ist ArrayList die richtige Antwort – das ist die Standardwahl, nicht der Notfall.

2 Die zehn Klassen

Klasse	Familie	Duplikate	Reihenfolge	Wofür
ArrayList	List	ja	wie eingefügt	Standard, wenn nichts gefordert ist
LinkedList	List	ja	wie eingefügt	viel Einfügen und Löschen am Anfang
HashSet	Set	nein	keine	nur Eindeutigkeit gefordert
LinkedHashSet	Set	nein	wie eingefügt	eindeutig UND Reihenfolge
TreeSet	Set	nein	sortiert	eindeutig UND sortiert
HashMap	Map	Keys eindeutig	keine	schnell über Schlüssel finden
LinkedHashMap	Map	Keys eindeutig	wie eingefügt	Schlüssel UND Reihenfolge
TreeMap	Map	Keys eindeutig	nach Key sortiert	Schlüssel UND sortiert
ArrayDeque	Queue	ja	FIFO	in Eingangsreihenfolge abarbeiten
PriorityQueue	Queue	ja	kleinstes zuerst	nach Priorität abarbeiten

Die Vorsilben bedeuten überall dasselbe

```
Hash...        keine Reihenfolge, dafuer am schnellsten
LinkedHash...  Einfuegereihenfolge bleibt erhalten
Tree...        sortiert
```

Du musst also nicht sechs Klassen einzeln lernen, sondern zwei Familien und drei Vorsilben.

3 Links Interface, rechts Klasse

```
Map<String, Car> m = new HashMap<>();
//  ^Interface          ^Klasse

List<Eruption> daten = new ArrayList<>();
Set<Book> ausgeliehen = new TreeSet<>();
Queue<Request> warteschlange = new ArrayDeque<>();
```

Map, List, Set und Queue sind Interfaces – reine Verträge. Davon kannst du kein Objekt erzeugen: new Map<>() gibt einen Fehler.

HashMap, ArrayList, TreeSet und so weiter sind die Klassen, die es tatsächlich umsetzen. Links schreibst du das Interface, rechts die Klasse.

4 Signalwörter im Aufgabentext

Im Aufgabentext steht	Antwort
„unter einer ID gespeichert"	Map
„effizientes Suchen anhand der ID"	Map
„schnell finden über den Namen"	Map
„keine Duplikate erlaubt" / „keine Dopplungen"	Set
„eindeutig"	Set
„sortiert vorliegen" / „natürlich sortiert"	Tree…
„nach dem Rückgabedatum sortiert"	Tree… mit Comparator
„in der Reihenfolge, in der sie hinzugefügt wurden"	LinkedHash… oder ArrayList
„in Einfügereihenfolge iterieren"	LinkedHash… oder ArrayList
„in Eingangsreihenfolge abarbeiten" (FIFO)	ArrayDeque als Queue
„nach Priorität abarbeiten"	PriorityQueue
„das mit der geringsten Haltbarkeit zuerst entnehmen"	PriorityQueue
„einem Schlüssel mehrere Werte zuordnen"	Map<K, List<V>>
„einer geeigneten Collection hinzufügen" (mehr nicht)	ArrayList

5 Beispiele aus echten Aufgaben

Beispiel 1 – CarPortal (Klausur SS 2019)

Aufgabentext

„Initialisiert die Datenstruktur cars. Diese speichert Car-Objekte unter einer ID. Wählen Sie die Datenstruktur so, dass die Car-Objekte in der Reihenfolge, in der sie hinzugefügt wurden, abgerufen werden können."

Herleitung

```
„unter einer ID"                          ->  MAP
„in der Reihenfolge, in der hinzugefuegt"     ->  LinkedHash...

Map<Integer, Car> cars = new LinkedHashMap<>();
```

Beispiel 2 – Dictionary (Klausur SS 2019)

Aufgabentext

„Wählen Sie eine Datenstruktur, die sicherstellt, dass die Elemente im Wörterbuch nach den Einträgen sortiert vorliegen."

Herleitung

```
Eintrag -> Uebersetzung, also Schluessel        ->  MAP
„sortiert vorliegen"                            ->  Tree...

TreeMap<K, V> eintraege = new TreeMap<>();
```

Beispiel 3 – Canvas (Klausur SS 2025)

Aufgabentext

„Implementieren Sie eine Klasse Canvas, die intern eine TreeMap<Point2D, Shape> nutzt."

Hier ist die Struktur vorgegeben – aber Achtung

Die TreeMap steht schon im Text. Der Haken liegt woanders: Point2D ist deine eigene Klasse. Damit eine TreeMap sortieren kann, muss der Schlüssel Comparable sein.

Genau deshalb steht in der Teilaufgabe davor: „Point2D Comparable machen". Die beiden Teilaufgaben hängen zusammen.

Beispiel 4 – Bibliothek (Übungsblatt 8)

Aufgabentext

„Verwenden Sie eine geeignete Collection, um das Ausleihkonto eines Nutzers zu verwalten. Sie soll die ausgeliehenen Bücher speichern, es sollen keine Duplikate erlaubt sein, und die Bücher sollen nach Rückgabedatum sortiert sein."

Herleitung

```
kein Schluessel genannt                         ->  kein MAP
„keine Duplikate erlaubt"                       ->  SET
„nach Rueckgabedatum sortiert"                  ->  Tree...
sortiert nach einem BESTIMMTEN Feld             ->  mit Comparator

Set<Book> ausgeliehen =
    new TreeSet<>(Comparator.comparing(Book::getReturnDate));
```

Verifiziert: Bei zwei identischen Büchern und einem weiteren bleiben zwei übrig, sortiert nach Datum.

Beispiel 5 – Vulkandaten (Übungsblatt 8)

Aufgabentext

„Lesen Sie die Daten ein und speichern Sie jeden Vulkanausbruch als Eruption-Objekt. Fügen Sie die Objekte anschließend einer geeigneten Collection hinzu."

Herleitung

```
kein Schluessel, keine Duplikatregel,
keine Sortierung gefordert                      ->  ArrayList

List<Eruption> daten = new ArrayList<>();
```

Hier gibt es nichts zu entscheiden – und deshalb steht in dieser Aufgabe auch keine Begründungsfrage. Wenn der Text nichts fordert, ist ArrayList richtig.

Beispiel 6 – Quersummen (Übungsblatt 7)

Aufgabentext

„Die Zeitstempel müssen so gespeichert werden, dass für eine gegebene Quersumme alle zugehörigen Zeitstempel effizient zugegriffen werden können. Es soll außerdem möglich sein, die Quersummen in aufsteigender Reihenfolge auszugeben. Hinweis: Sie müssen eine Map und eine Collection geeignet kombinieren."

Herleitung

```
„fuer eine gegebene Quersumme"                -> MAP
„aufsteigende Reihenfolge"                    -> Tree...
„ALLE zugehoerigen Zeitstempel"               -> mehrere Werte je Schluessel

Map<Integer, List<Long>> zeitstempel = new TreeMap<>();

// Eintragen, ohne vorher pruefen zu muessen:
zeitstempel.computeIfAbsent(quersumme, k -> new ArrayList<>())
           .add(System.currentTimeMillis());
```

computeIfAbsent legt die Liste an, falls der Schlüssel noch fehlt, und gibt sie zurück. Erspart die if-Abfrage.

6 Dieselben Daten, verschiedene Strukturen

Eingabe jeweils 3, 1, 3 – mit OpenJDK 21 ausgeführt

```
ArrayList       [3, 1, 3]      Duplikat bleibt, Reihenfolge bleibt
LinkedList      [3, 1, 3]      genauso
HashSet         [1, 3]         Duplikat weg, Reihenfolge nicht garantiert
LinkedHashSet   [3, 1]         Duplikat weg, Einfuegereihenfolge
TreeSet         [1, 3]         Duplikat weg, sortiert
```

Maps – eingefügt in der Reihenfolge b, a, c

```
HashMap         {a=1, b=2, c=3}    Reihenfolge zufaellig, hier nur zufaellig sortiert
LinkedHashMap   {b=2, a=1, c=3}    genau wie eingefuegt
TreeMap         {a=1, b=2, c=3}    nach Schluessel sortiert, garantiert
```

Wichtiger Unterschied

Bei der HashMap steht hier zufällig dasselbe wie bei der TreeMap. Verlassen darfst du dich darauf nicht – nur die TreeMap garantiert die Sortierung. Das ist der ganze Unterschied zwischen den beiden.

Queues – eingefügt 5, 1, 3

```
ArrayDeque      poll -> 5      FIFO: das zuerst Eingefuegte
PriorityQueue   poll -> 1      immer das kleinste Element
```

7 Was du zusätzlich brauchst

Wenn du das nimmst	brauchst du in deiner Klasse
HashSet oder HashMap mit eigenen Objekten	equals und hashCode
TreeSet oder TreeMap mit eigenen Objekten	Comparable mit compareTo, ODER einen Comparator im Konstruktor
PriorityQueue mit eigenen Objekten	Comparable ODER einen Comparator
ArrayList, LinkedList, ArrayDeque	nichts weiter

Die Merkregel für die Klausur

Sobald in einer Aufgabe TreeMap, TreeSet oder das Wort sortiert vorkommt und eine eigene Klasse beteiligt ist, ist Comparable die versteckte Pflichtaufgabe. Ohne sie fliegt zur Laufzeit eine

ClassCastException.

Genau darauf zielte SS 2025 mit „Point2D Comparable machen" direkt vor „Canvas mit TreeMap<Point2D, Shape>".

Comparator statt Comparable – wenn nach einem bestimmten Feld sortiert wird

```
Set<Book> s = new TreeSet<>(Comparator.comparing(Book::getReturnDate));
Queue<Request> q = new PriorityQueue<>(Comparator.comparingInt(Request::getPrio));
```

8 Kurzfassung zum Einprägen

```
Schluessel?      ja  ->  MAP
Duplikate?       ja  ->  LIST        nein ->  SET
Abarbeiten?          ->  QUEUE

Reihenfolge egal        ->  Hash...
wie eingefuegt          ->  LinkedHash...
sortiert                ->  Tree...

nichts gefordert        ->  ArrayList

Tree... + eigene Klasse ->  Comparable nicht vergessen!
Hash... + eigene Klasse ->  equals und hashCode nicht vergessen!
```